HERA: A Conversational Copilot for Identifying Ethical and Regulatory Risks in Health Apps

Nathan Massicot
MSE Data Science, Bern University of Applied Sciences
Quellgasse 21, 2502 Biel, Switzerland
nathan2massicot@gmail.com

Specialisation Project 2 — Supervisor: Prof. Dr. Kerstin Denecke

Abstract

The moment a piece of software touches health, it enters a maze of rules: data protection, medical-device law, artificial-intelligence regulation, accessibility. These frameworks overlap, change quickly, and differ from one country to the next, so ethical and regulatory risks are often noticed too late — after launch, when they are costly to fix and sometimes dangerous for patients. We present **HERA** (Health Ethical & Regulatory Assessment), a conversational copilot that helps a mobile-health (mHealth) app developer spot these risks early, right from the design stage. HERA rests on a new taxonomy that merges eight regulatory and ethical frameworks into seven pillars and thirty-eight dimensions. We turned official regulatory texts into short structured "cards", linked every taxonomy dimension to the rules that apply to it, and generated training dialogues from this knowledge base. On top of one small, locally-run language model we implemented and compared four architectures: a prompt-only baseline, a fine-tuned model (QLoRA), retrieval-augmented generation (RAG), and a multi-agent setup. All four were evaluated on twenty-five realistic app scenarios with a reproducible "simulated developer", automatic coverage metrics, and an LLM judge. RAG gave the most useful and best-grounded advice; the prompt-only baseline was a fast and solid fallback; the multi-agent setup was slower with no clear gain; and the fine-tuned model collapsed — a result we trace to data and hardware limits rather than to the method itself.

1 Introduction

The moment an app touches health, it enters a maze of frameworks. A single mobile health application can fall at once under data-protection law, the rules for medical devices, the new regulation on artificial intelligence, and accessibility standards. In Europe these include the General Data Protection Regulation [European Parliament and Council, 2016], the Medical Device Regulation [European Parliament and Council, 2017], the Artificial Intelligence Act [European Parliament and Council, 2024], and quality standards for health apps such as ISO/TS 82304-2 [International Organization for Standardization, 2021]. These texts overlap, they are revised often, and they differ from one country to the next: a health app sold in the European Union, in Switzerland and in the United Kingdom faces three partly different rule sets. In practice, more than ten regulatory frameworks may apply to one health app.

This is hard for the people who actually build the apps. Developers are neither lawyers nor ethicists, so it is difficult for them to know what actually applies to their product. The frameworks that could help already exist, but they are scattered and written for experts; none of them is designed to guide a developer, step by step, through the questions they should be asking. As a consequence, risks tend to surface late — after the product is live — when fixing them is expensive and, in a health setting, occasionally dangerous. What is missing is not another regulation but a tool that helps a developer think about these issues *upfront*, in a structured and reasonably complete way.

Our answer is a copilot that asks the right questions from the design stage. The system is built around **HERA** (Health Ethical & Regulatory Assessment), a taxonomy that unifies eight regulatory and ethical frameworks into a single, actionable structure that applies to health apps with or without AI. A conversational assistant uses this taxonomy to interview the developer one question at a time, dig into the specifics of their app, name concrete risks together with the precise regulation that applies, and finally produce a structured risk report. The idea of organising risk reasoning around an explicit, structured taxonomy and a small team of cooperating language-model "agents" is adapted from Deng et al. [2025], who applied it to ethi-

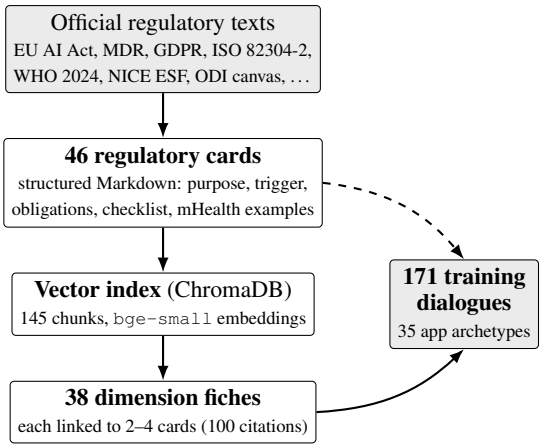


Figure 1: Knowledge-base construction. Official texts are summarised into regulatory cards, indexed in a local vector database, and linked to the taxonomy dimensions; the same material seeds the synthetic training dialogues.

cal awareness in finance; we re-target it at digital health.

This report makes four contributions. **(i)** We describe how we collected official regulatory texts and turned them into a small, structured knowledge base of regulatory cards (Section 2). **(ii)** We present the HERA taxonomy — seven pillars, thirty-eight dimensions — and the per-dimension "fiches" that link each dimension to the rules that govern it. **(iii)** We implement four copilot architectures on top of the *same* small, locally-run model and explain how each one was chosen, with its strengths and weaknesses (Section 3). **(iv)** We define a reproducible evaluation that compares the four approaches on twenty-five realistic scenarios using a simulated developer, automatic metrics, and an LLM judge (Section 4), and we report and discuss the results (Sections 5–6). Our headline finding is that retrieval-augmented generation gives the most useful advice, and that the disappointing fine-tuned model failed because of our hardware budget, not because the method is unsound.

2 Building HERA: from regulations to a structured knowledge base

Before any model can give advice, it needs the right knowledge in the right shape. This section describes how we gathered the regulatory material, how we organised it into the HERA taxonomy, how we linked each part of the taxonomy to the rules that apply, and how we produced the training dialogues. Figure 1 summarises the pipeline.

2.1 From scattered regulations to regulatory cards

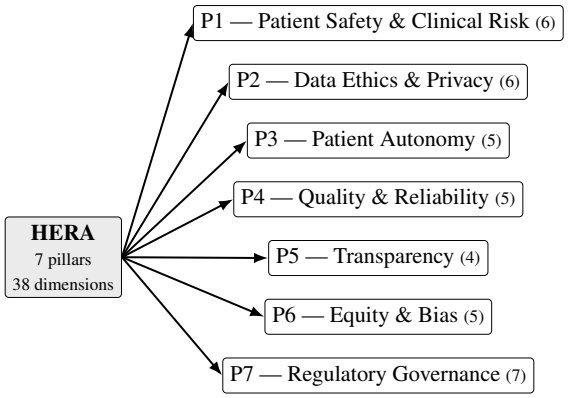
We started from the frameworks that practitioners already rely on. The selection was guided by the DTX Risk Assessment Canvas [Denecke et al., 2023], the World Health Organization guidance on AI for health [World Health Organization, 2024], the four principles of biomedical ethics [Beauchamp and Childress, 2019], and the main European legal texts [European Parliament and Council, 2024, 2017, 2016, International Organization for Standardization, 2021], complemented by the NICE evidence standards for digital health [National Institute for Health and Care Excellence, 2022]. In total we catalogued fourteen primary documents in a manifest (title, publisher, jurisdiction, key articles, official URL).

Reading fourteen long legal documents is exactly the burden a developer cannot carry, so we compressed the relevant content into **46 short "regulatory cards"**. Each card is a plain-text (Markdown) file with a fixed structure: what the rule is about, when it is triggered, the concrete obligations it imposes, a short checklist, a few examples specific to mHealth, and pointers to related cards. A card on the Medical Device Regulation, for instance, explains in a few lines when an app counts as a medical device, which class it is likely to be, and what that means in practice. The cards are the human-readable backbone of the whole system: they are easy to read, easy to keep up to date, and — as we will see — easy for a model to retrieve and cite.

2.2 The HERA taxonomy: seven pillars, thirty-eight dimensions

The cards tell a developer about individual rules, but they do not by themselves provide a map of *everything* that should be checked. That map is the HERA taxonomy. We organised the field into **seven pillars** — patient safety, data & privacy, patient autonomy, quality & usability, transparency, equity, and regulatory governance — and split each pillar into concrete **dimensions**, for a total of thirty-eight (Table 1). Crucially, every pillar applies to non-AI apps as well: a simple medication reminder still raises autonomy, privacy and reliability concerns without any algorithm.

Each dimension is described by a small "fiche" (Figure 2): a short identifier such as `[P2.D3]`, a name, a one-sentence plain-language description, a



Fiche — one dimension `[P2.D3]`
Name: Storage, Encryption & Security
Description: is health data protected at rest and in transit?
Reflection questions: where is data stored? is it encrypted? who can access it?
Typical risks: unencrypted records; weak access control
Mitigations: encrypt at rest/in transit; least-privilege access
Linked rules: GDPR Art. 9; GDPR Art. 32; `card_gdpr_art9`

Figure 2: Top: the HERA taxonomy — one root, seven pillars, thirty-eight dimensions (dimension count in parentheses). Bottom: the structure of a single dimension fiche, which links a taxonomy entry to the regulatory cards that apply to it.

handful of reflection questions, the risks that typically arise, and concrete mitigation ideas. This fixed shape is what lets the copilot move smoothly from a question ("where is the data stored?") to a named risk ("`[P2.D3]` unencrypted health data") to a usable mitigation ("encrypt at rest and document the lawful basis under GDPR Art. 9").

2.3 Linking dimensions to the rules that apply

We still needed a reliable bridge between an abstract dimension ("Storage, Encryption & Security") and the specific cards that govern it. Doing this by hand for thirty-eight dimensions would be slow and subjective, so we used *retrieval*. We indexed the 46 cards in a local vector database (ChromaDB), splitting each card into short overlapping passages and turning every passage into a numerical vector with a sentence-embedding model [Reimers and Gurevych, 2019, Xiao et al., 2024]. For each dimension we then searched the index using the dimension's name and reflection questions, and kept the closest cards. A short curation pass kept only the two to four cards that were genuinely relevant, each with a one-line explanation of *why* it applies. The result is a set of thirty-eight "source fiches" carrying one hundred citations in total — the machine-readable link between the taxonomy and the regulatory text.

2.4 Synthetic training dialogues

Finally, to be able to *train* a small model to behave like HERA, we needed examples of the target behaviour. We wrote thirty-five app "archetypes" (a diabetes tracker, a mental-health chatbot, an AI symptom checker, and so on, each tagged with its category, data sensitivity and target markets) and used them to generate **171 multi-turn dialogues**. In every dialogue a developer describes an app and the copilot asks reflection questions one at a time, citing dimension identifiers such as `[P1.D2]`, and closes with a short structured risk report. Each dialogue was checked against a strict format schema and de-duplicated. This set is the training material for the fine-tuned approach and the behavioural template that all four architectures imitate.

3 Copilot architectures

With the knowledge base in place, the central question of the project was: *what is the best way to turn this knowledge into good conversational advice?* We implemented four architectures and compared them head to head. Figure 3 shows the four side by side.

3.1 A shared backbone

To make the comparison fair, all four approaches share the same foundations. They run the *same* small open model, Qwen3-8B [Qwen Team, 2025], locally through Ollama, so no data ever leaves the machine. They sit behind one common software interface, so switching from one approach to another is a one-line change in the evaluation harness. And they start from the same compact HERA system prompt, which tells the model to ask exactly one focused question per turn, to cite the relevant dimension identifier when it raises a risk, and to pair each risk with one concrete mitigation and the precise regulation that applies. The four approaches differ only in *how* they use the HERA knowledge.

3.2 Prompt-only (baseline)

The simplest approach gives the model nothing but the HERA system prompt and lets it converse. It is the natural baseline: no training, no extra machinery, instant to set up, and very fast at run time. Its weakness is that it can only draw on what the base model already happens to know about regulation; its answers can be generic, and it may confidently cite an article that does not exist. It is, in effect, a knowledgeable generalist who has skimmed the

Pillar	Dimensions
P1 — Patient Safety & Clinical Risk	Clinical Evidence & Validation; Contraindications & Adverse Effects; False Sense of Security; Delayed Medical Consultation; Measurement Reliability; System Dependency & Over-reliance
P2 — Data Ethics & Privacy	Informed Consent; Data Minimization; Storage, Encryption & Security; Third-Party Data Sharing; Anonymization & Right to Erasure; Vulnerable Population Data
P3 — Patient Autonomy & Therapeutic Relationship	Right to Make Decisions; Emotional Manipulation & Excessive Nudging; Impact on Doctor–Patient Relationship; Dynamic Consent & Withdrawal Rights; Automation Bias
P4 — Quality, Usability & Technical Reliability	App Stability & Error Handling; Interoperability & Data Exchange; Accessibility (WCAG, literacy, language); User-Centered Design & Usability Testing; Offline Functionality & Updates
P5 — Transparency & Explainability	Understandable Output; Clear Communication of Limitations; Decision Traceability & Logging; Distinction Clinically-validated vs Estimated Content
P6 — Equity, Bias & Social Justice	Data & Clinical Rule Bias; Digital Divide & Socioeconomic Access; Cross-Cultural & Multilingual Validation; Impact on Existing Health Inequalities; Representativeness in Validation Studies
P7 — Regulatory Compliance & Governance	Medical Device Classification (SaMD); CE Marking & MDR Conformity; EU AI Act Risk Classification; GDPR Compliance (DPO/DPIA); Post-Market Surveillance & Vigilance; Third-Party Audit & Certification; Lifecycle & Documentation Management

Table 1: The HERA taxonomy: seven pillars and their thirty-eight dimensions.

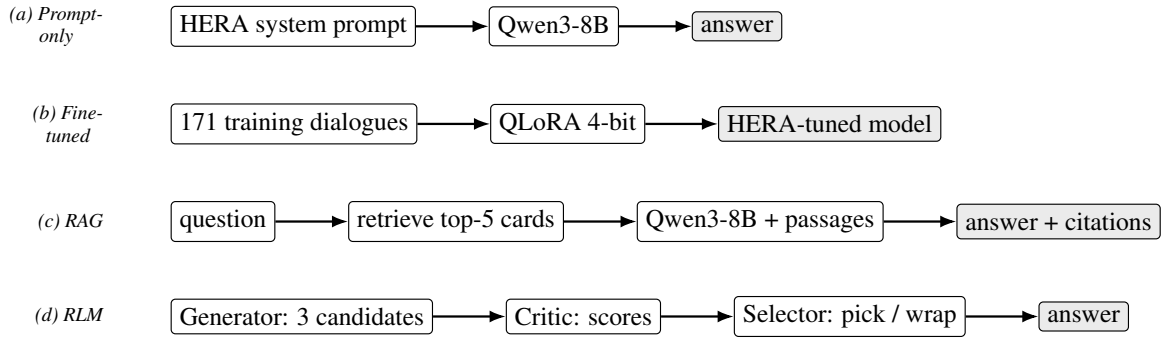


Figure 3: The four copilot architectures, all built on the same locally-run Qwen3-8B model. (a) prompt-only uses just the HERA system prompt; (b) the fine-tuned model bakes the HERA behaviour into the weights via QLoRA; (c) RAG retrieves and cites the relevant regulatory cards at every turn; (d) the multi-agent RLM proposes, critiques, and selects a question each turn (a separate synthesiser writes the final report).

topic but has no reference book open in front of them.

3.3 Fine-tuned model (QLoRA)

The second approach *teaches* the base model to behave like HERA by training it on the 171 dialogues. Full fine-tuning of an eight-billion-parameter model is far beyond a single consumer GPU, so we used QLoRA [Dettmers et al., 2023], which combines low-rank adapters [Hu et al., 2022] with 4-bit quantisation: instead of updating all the weights, it trains a small number of extra parameters on top of a compressed model. After training, the adapter is merged back and the model is again quantised to 4 bits so it can run locally. The appeal is that the HERA style becomes part of the model itself, with no retrieval needed at run time. The risk is equally clear: with little training data and heavy compression, quality can degrade — a risk that, as Section 5 shows, materialised.

3.4 Retrieval-augmented generation (RAG)

The third approach keeps the base model unchanged but gives it an open reference book. At every turn, the system takes the current conversation, retrieves the five most relevant regulatory cards from the vector index, pastes them into the prompt, and asks the model to ground its answer in them and to cite them [Lewis et al., 2020, Gao et al., 2023]. The advantages fit our problem well: answers are anchored in real regulatory text rather than in the model's memory, hallucinations are reduced, and the knowledge can be updated simply by editing a card — no retraining required. The cost is that quality depends on the retrieval step, and the longer prompts make each turn somewhat slower.

3.5 Multi-agent reasoning (RLM)

The fourth approach turns one turn of conversation into a little team effort, following the Generator/Critic/Selector design of Deng et al. [2025]

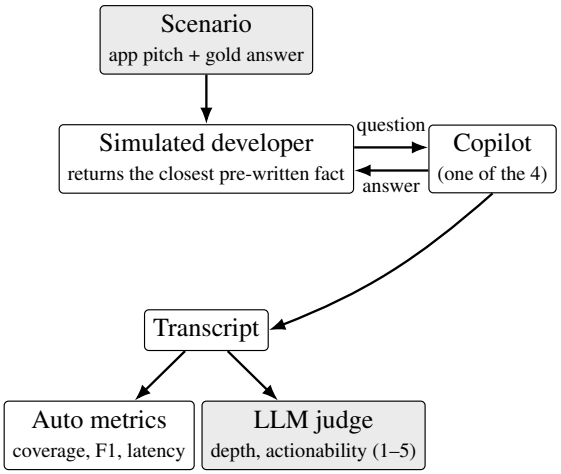


Figure 4: Evaluation protocol. Each of the four copilots talks to the same deterministic simulated developer over a scenario; the resulting transcript is scored both by automatic metrics and by an LLM judge.

and the broader idea of letting a model criticise and refine its own output [Shinn et al., 2023, Madaan et al., 2023]. A *Generator* proposes three candidate questions, each targeting a different, preferably uncovered, dimension. A *Critic* scores the three on relevance, depth and novelty. A *Selector* then either asks the strongest question or decides the conversation has covered enough and wraps up. A fourth role, the *Synthesiser*, writes the final risk report. The promise of "propose several, critique, then choose the best" is deeper and more on-target questions for the same base model. The price is that each turn now costs about three times as many model calls, and there are more moving parts that can fail.

4 Evaluation protocol

How do you decide which of four conversational systems gives better advice? Our guiding principle was that every approach must receive *exactly the same inputs*, so that we only ever compare how they reason. Figure 4 shows the set-up.

Scenarios. We wrote **25 realistic mHealth scenarios**, each with a short app description, a developer persona, and a hand-written "gold answer": the dimensions that should be raised, the risks that should be found (with a severity), the mitigations a good copilot would suggest, and the regulations that should be cited. The set spans all seven pillars, includes both AI and non-AI apps, and covers European and Swiss markets, so that no single approach can win just by specialising in one kind of app.

The simulated developer. The key methodological choice is how the "developer" answers. If we let a free language model improvise, each approach would receive different information and the comparison would be meaningless. Instead, the developer is *deterministic*: every scenario comes with a pool of pre-written facts, and when the copilot asks a question we embed it and return the single most relevant unused fact, measured by cosine similarity between the copilot's question and each useful, unused, stored answer, against a fixed threshold (0.60). So the returned answer is the closest one by cosine similarity: a sharper question lands on a more relevant fact. If nothing relevant matches — that is, if no stored answer exceeds the 0.60 similarity threshold — for several turns in a row, the conversation stops before it starts to loop.

Automatic metrics. From each transcript we compute four mechanical measures. *Coverage* is the share of the gold dimensions that the copilot actually raised (recall). *Relevance* is the share of the questions asked that touch a relevant dimension (precision). *Risk accuracy* is an F1 score on the named risks against the gold answer. *Latency* is the average time per turn. These are strict, regex-based counts: they are objective but unforgiving, and they penalise a useful answer the moment it mentions a dimension that the narrow gold list did not anticipate.

LLM judge. Because strict counting misses the real quality of advice, a separate language model acts as a judge [Zheng et al., 2023]. It reads each transcript — together with the gold dimensions — and rates it on two 1-to-5 scales: *Depth* (does the copilot drill into the specifics of *this* app and the precise article that applies, or stay generic?) and *Actionability* (could a competent developer implement the suggestions tomorrow, or are they vague platitudes?). The judge is explicitly told to be strict and not to inflate scores. The same simulated developer drives every approach, conversations run for up to ten to twelve turns, and the judge uses a low temperature for stability.

5 Results

Table 2 reports the four approaches on the twenty-five scenarios, on both the automatic metrics and the LLM judge.

The clearest message is that **RAG wins on the measures that matter most for usefulness**. It

Approach	Coverage	Relevance	Risk F1	Latency (s/turn)	Depth (1–5)	Actionability (1–5)
Prompt-only	0.322	**0.116**	**0.180**	**4.26**	2.48	2.20
Fine-tuned (QLoRA)	0.095	0.090	0.043	15.47	1.16	1.04
RAG	**0.359**	0.092	0.152	10.85	**3.04**	**2.72**
RLM (multi-agent)	0.261	0.062	0.087	20.72	2.40	2.08

Table 2: Evaluation results over 25 scenarios. *Coverage*, *Relevance* and *Risk F1* are automatic; *Depth* and *Actionability* are the LLM judge's averages. Best value in each column in bold (lower is better for latency).

achieves the best dimension coverage (0.36) and, more importantly, the best scores from the judge on both depth (3.0) and actionability (2.7): grounding each answer in the retrieved regulatory cards lets it refer to the right article and give concrete, implementable advice.

The **prompt-only baseline is surprisingly strong and by far the fastest** (4.3 seconds per turn, roughly two to five times quicker than the others). It even tops the strict automatic precision and F1, and lands second behind RAG on the judge. For a zero-setup approach this is a notable result, and it makes prompt-only a sensible fast fallback.

The **multi-agent RLM does not pay off here**. Its scores sit in the middle of the pack while its latency is the highest of all (20.7 seconds per turn, about three times the baseline), because every turn runs the model several times. The extra "propose, critique, select" machinery did not translate into better questions for this taxonomy and this base model.

The **fine-tuned model collapsed**. It is the weakest approach on every single measure, with judge scores close to the floor (depth 1.16, actionability 1.04). In its transcripts it repeats the same templated question turn after turn, invents references, and sometimes even renames the app. The training itself looked healthy — the validation loss fell steadily from 2.23 to 1.97 over three epochs with no sign of over-fitting — so the failure appears at inference time, not during learning. We return to this in the discussion.

It is worth noting that the automatic precision is low for *all* four approaches. This is largely an artefact of the evaluation: the gold list of dimensions is deliberately narrow, so any extra (and possibly still reasonable) dimension a copilot raises is counted as a mistake. This is precisely why we do not rely on the mechanical metrics alone, and why the LLM judge — which reflects the real quality of the advice — carries most of the weight in our reading of the results.

6 Discussion and conclusions

What we kept. Taken together, the results point to a clear choice. We kept the **RAG copilot** as the main system because it gives the best-quality, best-grounded advice and because its knowledge can be updated simply by editing a regulatory card, with the **prompt-only** model retained as a fast fallback. The multi-agent setup was dropped: it was markedly slower for no measurable gain. The end product is a small, fully local and private prototype — the model runs on the developer's own machine through Ollama and no data is sent to any external service — wrapped in a three-page web app (app profile, an interactive regulation diagram, and the HERA chatbot).

Why the fine-tuned model failed — and why it should not be dismissed. The collapse of the fine-tuned model deserves a careful reading, because it is a limitation of our *conditions*, not a verdict on the method. Two constraints stacked up. First, the training set was small: 171 dialogues is enough to nudge a model's style but thin for reshaping an eight-billion-parameter model. Second, and more decisively, we were limited by hardware. A full fine-tune of an 8B model needs far more GPU memory than we had available, so we could only afford QLoRA — a 4-bit adapter on top of an already compressed model — and we then quantised the merged model to 4 bits again so it would run locally. This double compression is the most likely cause of the mode collapse we observed (repeated questions, invented citations, empty reasoning tags). With a full- precision fine-tune on adequate hardware, more training data, and inference at higher precision, there is every reason to expect the fine-tuned model to become competitive with — and possibly stronger than — the prompt-only baseline. In other words, fine-tuning was not given a fair chance here; the result reflects our budget, not the ceiling of the approach.

Limitations. Beyond the hardware budget, three limitations temper our conclusions. The gold stan-

dard is narrow, which depresses the automatic precision and means those numbers should be read as trends rather than exact values. The LLM judge, while strict, is a single model, so its scores carry some variance. And the evaluation covers twenty-five scenarios: enough to see clear patterns, but not to split hairs over second-decimal differences. Finally, the current vector index covers the curated regulatory cards rather than the full source documents, which keeps retrieval focused but bounds how deep an answer can go.

Conclusion and future work. We set out to make the maze of digital-health regulation navigable for the people who build health apps. HERA unifies eight frameworks into an actionable taxonomy of seven pillars and thirty-eight dimensions, and a retrieval-grounded copilot built on a small, local model turns that taxonomy into useful, well-sourced advice given early, at design time. A rigorous comparison of four architectures over twenty-five scenarios shows that retrieval is the most useful approach today, that a plain prompt is a strong and fast fallback, and that fine-tuning was held back by hardware rather than by principle. The most promising next step is therefore to revisit fine-tuning with more data and a full-precision training run on better hardware, and to combine it with retrieval. Further work includes broadening the scenario set, using several judges to reduce variance, indexing the full regulatory documents, and running user studies with real developers to test whether a copilot like HERA genuinely changes how early risks are caught.

References

Tom L. Beauchamp and James F. Childress. *Principles of Biomedical Ethics*. Oxford University Press, 8 edition, 2019.

Kerstin Denecke, Richard May, Elia Gabarron, and Guillermo H. Lopez-Campos. Assessing the potential risks of digital therapeutics (DTX): The DTX risk assessment canvas. *Journal of Personalized Medicine*, 13(10):1523, 2023.

Deng, Luca Viganò, and Hauser. Using LLMs to foster ethical awareness, 2025. Generator/Critic/Selector multi-agent framework over a structured (HIEDE) risk taxonomy.

Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, and Luke Zettlemoyer. QLoRA: Efficient finetuning of quantized LLMs. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

European Parliament and Council. Regulation (EU) 2016/679 on the protection of natural persons with regard to the processing of personal data (general data protection regulation), 2016. Official Journal of the European Union.

European Parliament and Council. Regulation (EU) 2017/745 on medical devices (medical device regulation), 2017. Official Journal of the European Union.

European Parliament and Council. Regulation (EU) 2024/1689 laying down harmonised rules on artificial intelligence (artificial intelligence act), 2024. Official Journal of the European Union.

Yunfan Gao, Yun Xiong, Xinyu Gao, Kangxiang Jia, Jinliu Pan, Yuxi Bi, Yixin Dai, Jiawei Sun, and Haofen Wang. Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997*, 2023.

Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.

International Organization for Standardization. ISO/TS 82304-2:2021 health software — part 2: Health and wellness apps — quality and reliability, 2021.

Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

National Institute for Health and Care Excellence. Evidence standards framework (ESF) for digital health technologies, 2022.

Qwen Team. Qwen3 technical report, 2025.

Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2019.

Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

World Health Organization. Ethics and governance of artificial intelligence for health: Guidance on large multi-modal models. Technical report, World Health Organization, 2024.

Shitao Xiao, Zheng Liu, Peitian Zhang, Niklas Muennighoff, Defu Lian, and Jian-Yun Nie. C-Pack: Packed resources for general Chinese embeddings. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, 2023.

A Code and data repository

All source code, together with the HERA taxonomy and regulatory cards, the synthetic training dialogues, the 25 evaluation scenarios, and the evaluation harness, is openly available at:

https://github.com/Nathan-massicot/hera-risk-safety-copilot

The repository is organised as follows:

```
hera-risk-safety-copilot/
  src/                  core library
    copilot_base.py     shared interface + HERA prompt
    baseline/
      copilot.py        prompt-only copilot (Gradio)
      finetune.py       QLoRA fine-tuning
      inference.py      Ollama inference wrapper
    rag/
      ingest.py         build the ChromaDB index
      retriever.py      vector similarity search
      copilot.py        RAG-augmented copilot
    rlm/
      agents.py         Generator/Critic/Selector
      copilot.py        multi-agent copilot
    evaluation/
      runner.py         end-to-end orchestration
      simulator.py      deterministic developer
      metrics.py        coverage / F1 / latency
      llm_judge.py      LLM-as-judge
  data/
    taxonomy/           taxonomy + dimension fiches
    regulations/        46 regulatory cards
    training/           171 synthetic dialogues
    evaluation/         25 scenarios + judge ratings
  configs/              model / training / eval YAML
  scripts/              data-gen, ingestion, rendering
  models/               LoRA adapter, merged, ChromaDB
  web/                  FastAPI + React 3-page app
```